

Table des matières

1. Premiers pas avec Yacc
2. Gestion des erreurs et suppression des conflits
3. Variables et affectations
4. Explications des fichiers Lex et Yacc
5. Table des symboles

1. Premiers pas avec Yacc

Le but de cet exercice est d'écrire une calculatrice. L'utilisateur saisit l'opération composée d'additions et/ou de soustractions d'entiers et la termine par un « . » (un point).

1. Téléchargez le code d'exemple : [exemple.zip](#). [Cliquez ici](#) si vous souhaitez avoir plus d'informations sur le fichier.
2. Compilez le programme (ignorez les conflits pour le moment) et vérifiez son comportement : saisissez « 1+2. », puis tapez sur la touche « entrée » pour obtenir le résultat. Pressez CTRL+D pour terminer le programme.
3. Que se passe-t-il si vous saisissez une opération plus complexe (avec plusieurs opérateurs) ? Ou deux opérations différentes ?

Pour permettre plusieurs opérations de suite, il est nécessaire de modifier la grammaire. Pour le moment, la première règle indique qu'un EXEC est une EXPRESSION suivie d'un « . ».

4. Comment modifier la grammaire pour accepter plusieurs opérations de suite ?
5. Les résultats s'affichent-ils bien au fur-et-à-mesure ?

Attention ! Vous ne devez pas ajouter de nouveaux conflits lors de la modification de la grammaire.

2. Gestion des erreurs et suppression des conflits

Nous souhaitons ajouter les opérateurs « * » et « / ». Pour gérer le problème de la division par zéro, il faut considérer que l'erreur peut se produire à tout moment dans une opération et qu'il n'est pas possible d'utiliser uniquement la valeur de `$$` (la valeur -1, par exemple, peut correspondre au résultat de l'opération « 1 - 2 »). Une solution simple est d'utiliser une variable globale `error` qui prend la valeur `TRUE` ou `FALSE`.

1. Où déclarer cette variable ? Quand l'initialiser et la modifier ?
2. Modifiez votre grammaire pour prendre en compte les deux nouveaux opérateurs, en gérant les erreurs liées à la division par zéro.
3. Vérifiez qu'après une erreur, le calcul suivant se déroule correctement.

En compilant les fichiers de l'exercice précédent, *Yacc* nous informe que le programme comporte des conflits décalages/réductions (4 si vous n'en avez pas ajouté).

4. Pour visualiser les conflits, ajoutez l'option « v » dans la constante `YACCFLAGS` du `makefile` (remplacez « -d » par « -vd »). En recompilant, le fichier `y.output` est normalement généré. Ouvrez-le et analysez-le.
5. Essayez les calculs « 1+2*3 » et « 2*3+1 ». Quels sont les résultats ? Pourquoi ?
6. Quelles sont les deux méthodes vues en cours pour corriger ce problème ?

Nous souhaitons utiliser la solution proposée par *Yacc* avec la directive `%left` (voir le cours).

7. Modifiez votre fichier *Yacc* en ajoutant la priorité des opérateurs.
8. Vérifiez qu'il n'y a plus de conflits et que les opérations sont maintenant justes, mathématiquement parlant.

Pour gérer les nombres négatifs, nous souhaitons modifier le fichier *Lex*.

9. Modifiez le modèle permettant de reconnaître un entier pour reconnaître un entier négatif (pour rappel, le « ? » permet de rendre optionnel ce qui précède).
10. Faites les opérations « -1+2 » et « 2-1 ». Que se passe-t-il ?

Comme *Lex* essaie de reconnaître le plus grand lexème possible, -1 est vu maintenant comme un entier, d'où l'erreur affichée quand *Yacc* tente d'analyser l'entrée. Une solution est de reconnaître directement le « - » comme un opérateur unaire.

11. Supprimez les modifications dans le fichier *Lex*.
12. Ajoutez une nouvelle règle pour `EXPRESSION` pour reconnaître le moins unaire.
13. Vérifiez que toutes les opérations sont maintenant fonctionnelles.

3. Variables et affectations

Pour cet exercice, nous avons besoin de la table des symboles dont le développement a été demandé à la fin du TP précédent. [Cliquez ici](#) pour plus d'informations sur la table des symboles.

Nous souhaitons pouvoir ajouter la possibilité de sauvegarder un calcul dans une variable et d'utiliser cette dernière dans des calculs. La calculatrice doit accepter maintenant des opérations classiques et des affectations. Par exemple « $x = 1 + 2$. » peut être suivi par « $1 + x$. » (toujours avec le « . » final). Le résultat de l'affectation est également affiché à l'écran.

Attention ! Nous allons devoir faire beaucoup de modifications. Il est conseillé de faire une sauvegarde de votre code.

L'analyseur lexical va devoir reconnaître des chaînes de caractères (une suite de lettres en majuscules/minuscules) qui pourront être analysées soit comme un identificateur (un nom de variable inconnu), soit comme une variable (un pointeur vers un symbole de la table des symboles). Pour le moment, *Yacc* permet de retourner uniquement des entiers. Pour modifier cela, nous devons utiliser la directive `%union`.

1. Avec la directive `%union` dans le fichier *Yacc*, définissez 3 champs : `value` de type `int`, `ptr` de type `char[MAX_NAME]` (où `MAX_NAME` est la longueur maximum d'un nom dans votre table des symboles) et `symbol` de type `symbol_t` (un symbole de votre table des symboles).
2. Modifiez la directive `%token` pour `integer` et spécifiez le type `value`.
3. Ajoutez les symboles terminaux `variable` de type `symbol` et `id` de type `ptr`.
4. Grâce à la directive `%type`, spécifiez le type `value` pour les symboles non terminaux `EXPRESSION` et `ASSIGNMENT`.
5. Ajoutez deux règles pour `ASSIGNMENT` : une pour l'initialisation d'une variable (l'affectation d'une `EXPRESSION` à un identificateur) et une pour l'affectation d'une variable (l'affectation d'une `EXPRESSION` à une variable existante). La première règle ajoute le symbole dans la table des symboles puis retourne la valeur affectée. La seconde modifie la valeur du symbole et la retourne.
6. Ajoutez une nouvelle règle pour `PROGRAM` pour ajouter la possibilité d'un `ASSIGNMENT`.

Il reste maintenant à modifier le fichier *Lex*. Il faut d'abord supprimer la déclaration de `yy1val` (qui n'est plus un entier) dans la partie déclaration. Elle n'est plus nécessaire car l'utilisation de la directive `%union` entraîne une déclaration de `yy1val` (une union contenant les champs spécifiés dans le *Yacc*) dans le fichier *y.tab.h*.

7. Modifiez la règle pour les entiers : la valeur est placée dans le champ `value` de `yy1val`.
8. Ajoutez une règle pour les identificateurs. On recherche dans un premier temps `yytext` dans la table des symboles. Si aucun symbole ne correspond, on copie la valeur de `yytext` dans le champ `ptr` de `yy1val` puis on retourne un `id`. Sinon, le champ `symbol` correspond au symbole de la table des symboles.

Il reste maintenant à gérer la table des symboles. Dans une calculatrice, c'est assez simple : il suffit de déclarer une variable globale et de l'initialiser (dans le `main` par exemple). Il faudra bien sûr ajouter les inclusions dans les fichiers *Lex* et *Yacc*, ainsi que la déclaration (avec `extern`) de la variable.

9. Ajoutez la déclaration de la table des symboles dans le fichier *calculator.c*.
10. Ajoutez les inclusions (*table.h*) et la déclaration de la table des symboles dans les fichiers *Lex* et *Yacc*.
11. Compilez et vérifiez que votre calculatrice fonctionne correctement.

4. Explications des fichiers Lex et Yacc

Le fichier *Lex*

Il est très similaire à celui que nous avons vu dans le TP précédent. Nous avons ajouté la déclaration de la fonction `yyerror` qui est définie dans la partie C (tout en bas du fichier). Elle permet d'afficher des erreurs lors de l'analyse (lexicale ou grammaticale). Nous déclarons également la variable `yy1val`. Elle permet à *Lex* de transférer les valeurs (les lexèmes) à *Yacc*. Par défaut, son type est `int`.

La partie règles comporte 4 règles. La première correspond aux entiers (le modèle n'est pas parfait, mais il est attendu qu'il soit amélioré pour [le projet](#)). Dès qu'un entier est détecté, on convertit la valeur de `yytext` en entier et on l'affecte à la variable `yy1val`. La deuxième règle correspond aux opérateurs et au point (utilisé comme séparateur). Ces caractères sont envoyés directement à *Yacc*. La troisième règle permet d'ignorer tous les espaces (ainsi que les tabulations, les caractères non affichables tels que `\r`). La dernière règle permet d'afficher un message d'erreur pour tous les autres caractères rencontrés.

Le fichier *Yacc*

Il est constitué de 3 parties (comme le fichier *Lex*) : la partie déclaration C (entre `%{` et `%}`), la partie déclaration *Yacc* et la partie des règles entourée par `%%`.

Dans la partie déclaration C, nous avons déclaré la fonction `yyerror` qui nous permettra d'utiliser cette fonction lors de l'analyse grammaticale.

Dans la partie déclaration *Yacc*, nous avons simplement déclaré le symbole terminal (*token* en *Yacc*) `integer`. La directive `%token` permet de spécifier également un type, mais ce n'est pas nécessaire pour le moment (par défaut, ce sera un entier).

La partie règles comporte 5 règles : 2 pour `EXEC` et 3 pour `EXPRESSION`.

La première règle définit ce qu'est une `EXEC` : une `EXPRESSION` suivie d'un point. Le `:"` correspond à la flèche que nous utilisons en TD pour séparer les parties droite et gauche des règles de grammaire. Les accolades permettent de décorer la grammaire : il s'agit de code C qui est exécuté. Ici, il est exécuté lorsque la règle est réduite. Le symbole de la partie gauche est identifié par `$$` et chaque symbole de la partie droite de la règle est identifié par `$1 ... $n`. Dans la partie C, il est possible de récupérer les valeurs associées (le `yy1val`) grâce à ces identificateurs. Ici, nous affichons la valeur `$1`, ce qui correspond à la valeur de `EXPRESSION`.

Le `|` correspond à la seconde règle. Comme il n'y a rien, cela correspond à la règle `EXEC` $\rightarrow \epsilon$. Si on démarre la calculatrice et qu'on l'arrête immédiatement (avec CTRL+D) cela ne génère pas d'erreur.

Il y a 3 règles associés à `EXPRESSION`. La première permet de reconnaître le symbole terminal `integer`. On remarque qu'il n'y a pas de décoration. En son absence, *Yacc* ajoute simplement l'instruction `$$ = $1`. L'expression sera donc égale à la valeur de l'entier. Les deux autres règles sont similaires : on récupère les deux opérandes et on effectue l'opération en fonction de l'opérateur. La valeur de l'expression est le résultat de l'opération.

Le programme

Comme nous devons lancer l'analyse grammaticale dans le `main`, nous lançons la fonction `yyvsparse` (au lieu de `yylex` dans le TP précédent). Elle ne prend pas de paramètres et retourne un entier. Il est nécessaire de la déclarer.

5. Table des symboles

La table des symboles est utilisée par *Lex* et *Yacc* pour stocker tout un ensemble d'éléments (nommés symboles) : des variables, des déclarations de fonctions, des types, *etc.*

Un symbole est caractérisé par un nom, un type (variable, fonction, *etc.*) et d'autres attributs qui dépendent de son type. Par exemple, pour une variable, nous avons besoin en plus de son type de valeur (int, float, *etc.*). Dans le cas de spécifique de la calculatrice, une variable n'a pas besoin de type de valeur mais d'une valeur (ici, un entier).

Une table des symboles utilise une table de hachage, ce qui permet un accès plus rapide aux différents symboles. Pour simplifier, il ne s'agit pas d'une liste chaînée de symboles mais d'un tableau de N listes chaînées de symboles. Pour savoir dans quelle liste stocker ou chercher un symbole, on utilise une fonction de hachage qui convertit le nom du symbole en un entier compris dans l'intervalle [0; N-1].

Pour écrire une table des symboles, nous avons besoin des structures suivantes (les noms sont donnés à titre indicatif) :

- Un symbole (par exemple `symbol_t`)
- Une liste chaînée de symboles (par exemple `lst_symbol_t`)
- Une table (par exemple `table_t`)

La structure `lst_symbol_t` n'est pas nécessaire si chaque `symbol_t` possède un pointeur vers le `symbol_t` suivant.

Pour chaque structure, nous avons besoin des fonctions suivantes (les noms sont donnés à titre indicatif) :

- symbole : `symbol_create`, `symbol_display`, `symbol_delete`
- liste chaînée de symboles : `lst_symbol_create`, `lst_symbol_display`, `lst_symbol_delete` et `lst_symbol_search`
- table des symboles : `table_create`, `table_display`, `table_delete`, `table_add`, `table_search`
- hash (la fonction de hachage)